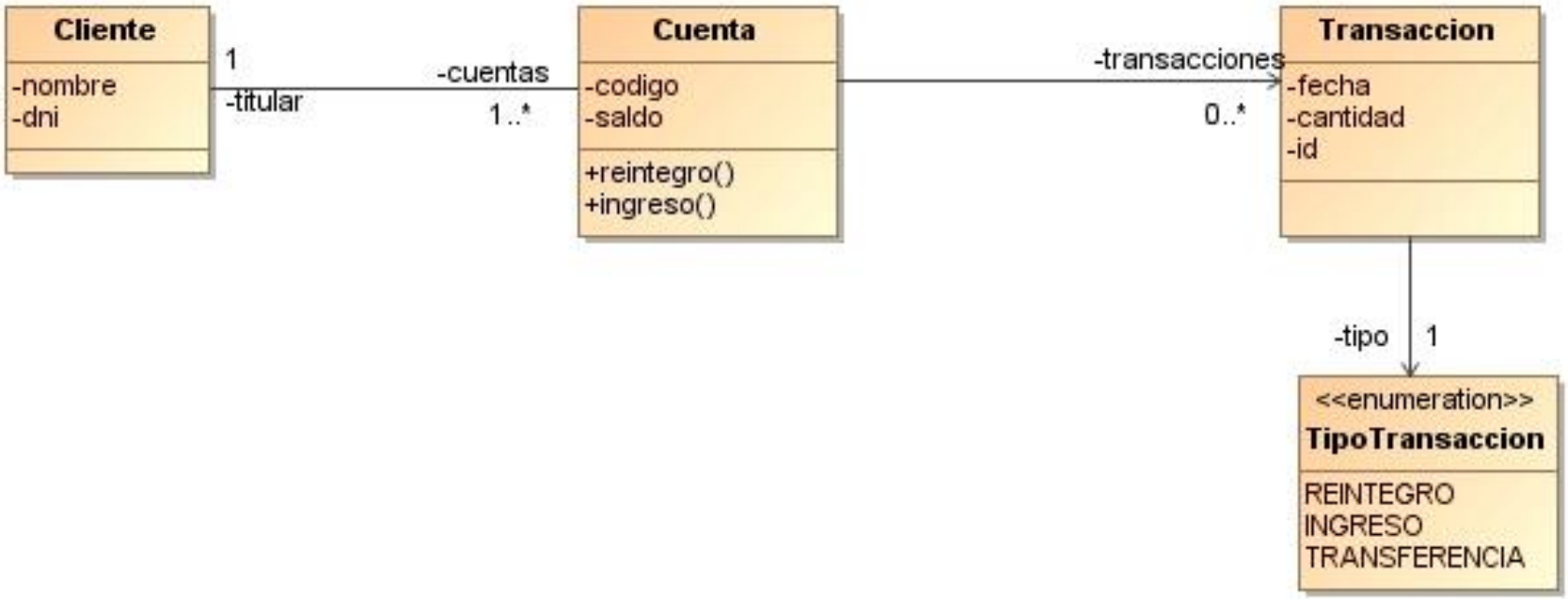
Ejercicios de manejo de Stream



1. Dada una lista de cuentas, retornar una lista de aquellas cuentas que tienen el saldo mayor de una cantidad y ordenadas por el valor del saldo de mayor a menor.

```
List<Cuenta> alc = cuentas.stream()
        .filter(cu-> cu.getSaldo()>=1000)
        .sorted(comparing(Cuenta::getSaldo).reversed())
        .collect(Collectors.toList());
```

2. Dada una lista de cuentas, imprimir por consola el número de cuentas que tienen al menos una transacción.

```
long num = cuentas.stream()
           .filter(cu -> !cu.getTransacciones().isEmpty())
           .count();
```

3. Dado una lista de cuentas, incrementar un 1% el saldo de las cuentas cuyo saldo es mayor de cierta cantidad, realizando un ingreso con la correspondiente cantidad.

```
cuentas.stream()
        .filter(cu ->cu.getSaldo()>cantidad)
        .forEach(cu ->
              cu.ingreso(cu.getSaldo()*0.01);
```

4. Dada una lista de cuentas, imprimir por consola los DNIs de los clientes cuyas cuentas superen una cantidad.

```
cuentas.stream()
    .filter(c -> c.getSaldo() > cantidad)
    .map(c -> c.getTitular().getDNI())
    .distinct()
    .forEach(System.out::println);
```

5. Dada una lista de cuentas, obtener la lista ordenada de los códigos de las cuentas que tienen, al menos, una transacción cuya cantidad es mayor que determinado valor.

```
List<Strings> codigos = cuentas.stream()
    .filter(cu ->
            cu.getTransacciones().stream()
                .anyMatch(t -> t.getCantidad() > cantidad))
    .map(c -> c.getCodigo())
    .sorted()
    .collect(toList());
```

6. Dada una lista de cuentas, imprimir por consola los códigos, nombres de titular y número de transacciones de tipo reintegro en un año dado, de las cuentas que hayan realizado más de cierto número de reintegros en ese año.

```
Function<Cuenta,Long> numTransAño = (cuenta ->
cuenta.getTransacciones().stream()
          .filter(tran -> tran.getTipo() == TipoTransaccion.REINTEGRO
                          && trans.isAño(AÑO)).count());
cuentas.stream()
    .filter(cuenta -> numTransAño.apply(cuenta) >  limiteReintegros)
    .forEach(cuenta ->
               System.out.println(
               cuenta.getCodigo() + "   " +
               cuenta.getTitular() +
               numTransAño.apply(cuenta))
```

7. Dada una lista de cuentas, obtener una lista de los identificadores de las transacciones cuya cantidad es mayor que un determinado valor.

```
List<String> ids = cuentas.stream()
    .flatMap(cu->cu.getTransacciones().stream())
    .filter(t -> t.getCantidad()>valor)
    .map(Transaccion::getID)
    .collect(toList())
```

8. Dada una lista de clientes, obtener una lista de los nombres de clientes que tienen al menos una cuenta superior a cierta cantidad de dinero.

```
List<String> nombresDeClientes = clientes.stream()
        .filter(cliente -> cliente.getCuentas().stream()
            .anyMatch(cuenta -> cuenta.getSaldo() > cantidadLimite))
        .map(Cliente::getNombre)
        .distinct()
        .collect(Collectors.toList());
```

Otra opción:

```
List<String> nombresDeClientes = clientes.stream()
    .flatMap(cliente -> cliente.getCuentas().stream())
    .filter(cuenta -> cuenta.getSaldo() > cantidadLimite)
    .map(cuenta -> cuenta.getTitular().getNombre())
    .distinct()
    .collect(Collectors.toList());
```

9. Dada una lista de clientes, escribe un código que retorne una lista de aquellos clientes que tienen una suma total en sus cuentas que supera cierta cantidad.

```
List<Cliente> listaclientes = clientes.stream()
    .filter(cliente -> cliente.getCuentas().stream().
      .collect(Collectors.summingDouble(Cuenta::getSaldo))>cant)
    .collect(Collectors.toList());
```

Otra opción:

```
List<Cliente> listaclientes = clientes.stream()
        .filter(cliente -> cliente.getCuentas().stream()
            .mapToDouble(cuenta -> cuenta.getSaldo())
            .sum() > cantidad)
        .collect(Collectors.toList());
```

10. Dada una lista de clientes, retornar una tabla que asocie el dni de los clientes con la suma total de dinero en sus cuentas si el cliente posee más de una cantidad dada.

```
List<Cliente> lc = clientes.stream()
    .filter(cli -> cli.getCuentas().stream()
      .collect(Collectors.summingInt(Cuenta::getSaldo))>cant)
    .collect(toMap(Cliente::getDNI,
              cli -> cli.getCuentas().stream()
                   .collect(Collectors.
                        summingInt(Cuenta::getSaldo))))
```

Otra opción:

```
List<Cliente> lc = clientes.stream()
    .filter(cli -> cli.getCuentas().stream()
              .mapToDouble(cuenta -> cuenta.getSaldo())
             .sum() > cantidad)
    .collect(toMap(Cliente::getDNI,
              cli -> cli.getCuentas().stream()
                  .mapToDouble(cuenta -> cuenta.getSaldo())
                  .sum()));
```

Function <Cliente,Double>

10. Dada una lista de clientes, escribe un método que retorne una tabla que asocie el dni de los clientes con la suma total de dinero en sus cuentas si el cliente posee más de una cantidad dada.

```
Function <Cliente,Integer> sumaDineroCliente =
        cli -> cli.getCuentas().stream()
                  .mapToInt(Cuenta::getSaldo())
                  .sum()
```

```
clientes.stream()
    . filter( cli -> sumaDineroCliente.apply(cli) > cantidad)
    . collect(toMap(cli -> cli.getDNI(),
                    sumaDineroCliente.apply(cli))
```